Testbericht 16 – Text und SVG-Import

Plan 17.7, Schritt 3 (ohne eigene Schablonenschrift): `text()` mit mitgelieferten OFL-Schriften, `import()` von SVG · neue Crates cad-outline, cad-svg, cad-font; cad-eval, cad-ir, cad-engine (Farbregionen), cad-step (Ellipsen), SMLib-Fassade (getrennte Vereinigung), cad-cli, cad-desktop, web · 23.10.2026 · Branch worktree-agent-a9b869d2499e391b9 · Commit 6774bb4 · rustc 1.97.0 (2d8144b78 2026-07-07) · 4 Kerne

cargo test --workspace	clippy · fmt	Vitest · Playwright	OpenSCAD Text	Wasm cad_wasm	„Hello World“
520 / 520	**sauber · sauber**	**220 · 102 / 103**	**≤ 0,004 mm**	**+2,17 MB**	**505 ms**
davon 42 neu/geändert	-D warnings	neu: T01–T05; C16 zeitabhängig	Hülle; Fläche ≤ 0,28 %	3,97 MB, gzip 1,70 MB	Kernel nativ; Browser 918 ms

1 Auftrag und Ergebnis

Aus Schritt 3 von Plan 17: **SVG-Import** (Teilmenge, 17.5) und **Text** mit mitgelieferten freien Schriften (17.4). Die eigene Schablonenschrift `"cad stencil mono"` gehört nicht dazu (sie braucht `stroke` aus Schritt 2); der Name ist als künftige Standardschrift reserviert, bis dahin ist **JetBrains Mono** die Standardschrift (Hinweis bei `font = "cad stencil mono"`).

- `text(t, size, font, halign, valign, spacing, direction, language, script)`: Umrisse exakt (TrueType quadratisch, CFF kubisch), Kerning und Ligaturen über `rustybuzz`, Löcher nach nonzero, `color()` wie sonst. Hülle und Fläche gegen OpenSCAD 2021.01 mit denselben Schriftdateien: Hülle ≤ 0,004 mm, Fläche ≤ 0,28 % (OpenSCAD tastet die Kurven ab).
- `import("….svg", id, scale, dpi = 96, center)`: Pfade (alle Befehle), Rechtecke mit Rundung, Kreise und Ellipsen exakt, Polygone, Gruppen und verschachtelte `svg` mit `transform`, Einheiten, `fill-rule`, `display:none`; alles andere mit Warnung und SVG-Zeile.
- Dateien relativ zur einbindenden Datei (CLI: Dateisystem; Browser: Projektdateien; Desktop: liest und schreibt sie); geänderte SVGs ändern den Cache-Schlüssel, `cad watch` folgt ihnen, die UI-Szene auch (T02).
- Pflicht-Zeichensatz mit Umlauten, ß/ß und Satzzeichen in allen mitgelieferten Schriften (Test); Lücken (nur Allerta Stencil) kommen mit Warnung aus JetBrains Mono.
- Nebenbei: Farbregionen von Modellen, die nur Vereinigungen ihrer Farbknoten sind, ohne Boolean mit dem Ganzen (mehrfarbiger Text: Sekunden statt Abbruch durch den Watchdog).
- Gefunden mit der Alphabet-Tafel: SMLib vereinigte getrennte Glyphenteile falsch (Allerta-„c“: ein Stück doppelt, einmal verkehrt herum; Volumen 2,0 statt 7,3 mm³, „a“ in „abcd“ verschwand, ohne Meldung). Die Fassade vereinigt Körper mit getrennten Hüllquadern jetzt ohne SMLib-Boolean (`disjoint_union`: beide Schalen in eine Brep); Regressionstest `text_volume_is_area_times_height`.

2 Entscheidungen

Frage	Entscheidung
Bibliotheken	`roxmltree` 0.21 (XML, Zeilen für Meldungen), `rustybuzz` 0.20.1 mit `ttf-parser` 0.25 (Shaping wie HarfBuzz, Umrisse, reines Rust, Wasm-tauglich). Pfaddaten, Transformationen, Einheiten, Bögen, Füllregeln: eigener Code.
Aufteilung	Neue Crates `cad-outline` (neutrale Konturen), `cad-svg`, `cad-font`; in `cad-eval` nur die Builtins (`import.rs`) und das Laden der Datendateien (`Project::assets`). IR: `Seg::Nurbs` (aus Schritt 2) für B-Spline-Ketten und Ellipsenbögen. Engine unverändert außer den Farbregionen; Fassade: Vereinigung getrennter Körper ohne SMLib-Boolean (unten).
Profil-Knoten	Ein `Profile::Sketch` je gefülltem SVG-Element bzw. je Glyphe, mehrere unter einer Gruppe (Vereinigung): überlappende Elemente oder Glyphen ergeben die Vereinigung wie beim Malen; innerhalb eines Elements entscheidet seine Füllregel.

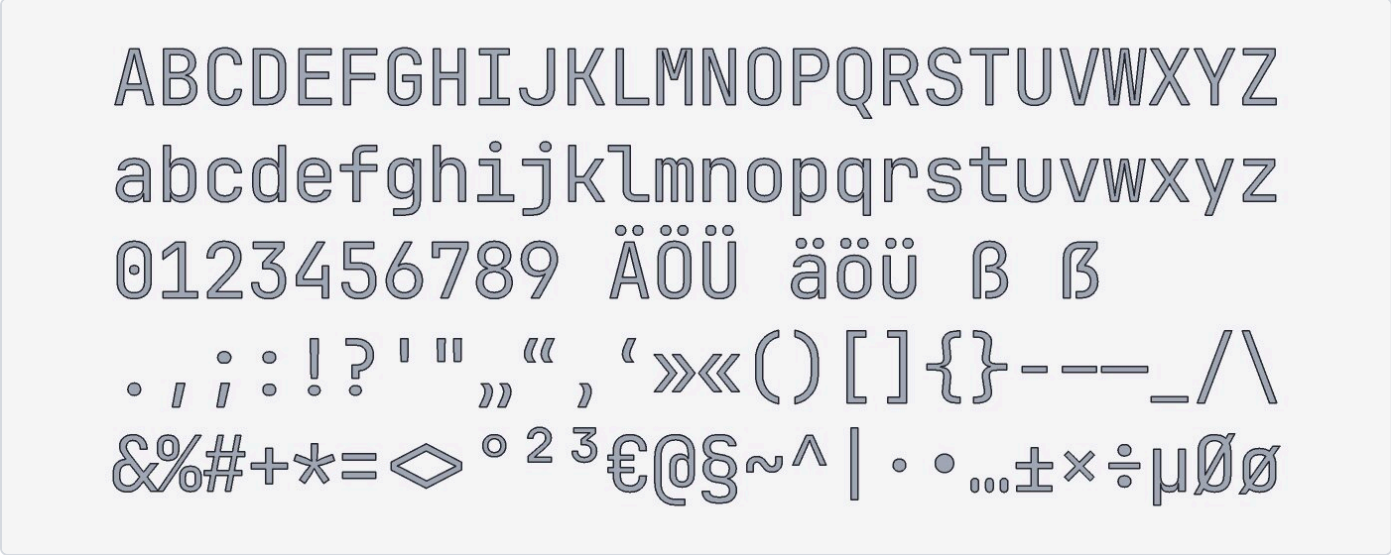
Kurven	Tangentenstetige Ketten von Bézierkurven einer Glyphe oder eines Pfads werden **eine** B-Spline (Knoten C1, Parameterlängen aus den Tangentenlängen; exakt), Ecken trennen. Weniger Kernel-Flächen (Glyphe „o“: 4 statt ~30).
Größe `size`	wie OpenSCAD: em = `size`/0,72 (OpenSCAD setzt die Schrift auf `size` Punkt bei 100 dpi); Versalhöhe ≈ `size`. Laufweite, Versatz und Ausrichtung mal 1000/1024 wie OpenSCAD (dessen FreeType-Umrechnung): Text bleibt gleich breit wie in OpenSCAD.
Ausrichtung	`halign` nach der Laufweite, `valign` nach der Glyphenhülle (top/center/bottom), `baseline` 0; senkrechter Text mit den Versätzen des Shapers (wie OpenSCAD zentriert).
Standardschrift	JetBrains Mono Regular, bis `"cad stencil mono"` existiert (Name reserviert, Hinweis). Unbekannte Familie: Warnung + JetBrains Mono; unbekannter Stil: Warnung + Regular der Familie.
Fehlende Zeichen	Rückfall je Zeichen auf JetBrains Mono gleichen Gewichts (OS/2-Gewicht ≥ 600 → Bold) mit Warnung am Statement: ein lesbares Zeichen an seiner Stelle ist nützlicher als ein Kästchen oder eine Lücke, die beim Drucken übersehen wird; JetBrains Mono deckt den Pflicht-Zeichensatz und weit mehr ab. Allerta Stencil bleibt unverändert (Reserved Font Name „Allerta“: veränderte Fassungen dürften nicht so heißen).
SVG-Einheiten	wie OpenSCAD, wo es konsistent ist; Standard `dpi = 96` (OpenSCAD 2021.01: 72 für Zahlen ohne Einheit). Abweichungen siehe 5.3.
Dateien	Jeder Text im Projekt, der auf `.svg/.ttf/.otf/.ttc/.otc` endet, wird beim Laden gelesen (auch Parameter-Standardwerte); Binärdateien liegen in Text-Projekten (Browser, `SetFiles`) als `data:`-URL. Ein zur Laufzeit zusammengesetzter Name, der nirgends als Text steht: Fehler „nicht geladen“.

3 Schriften

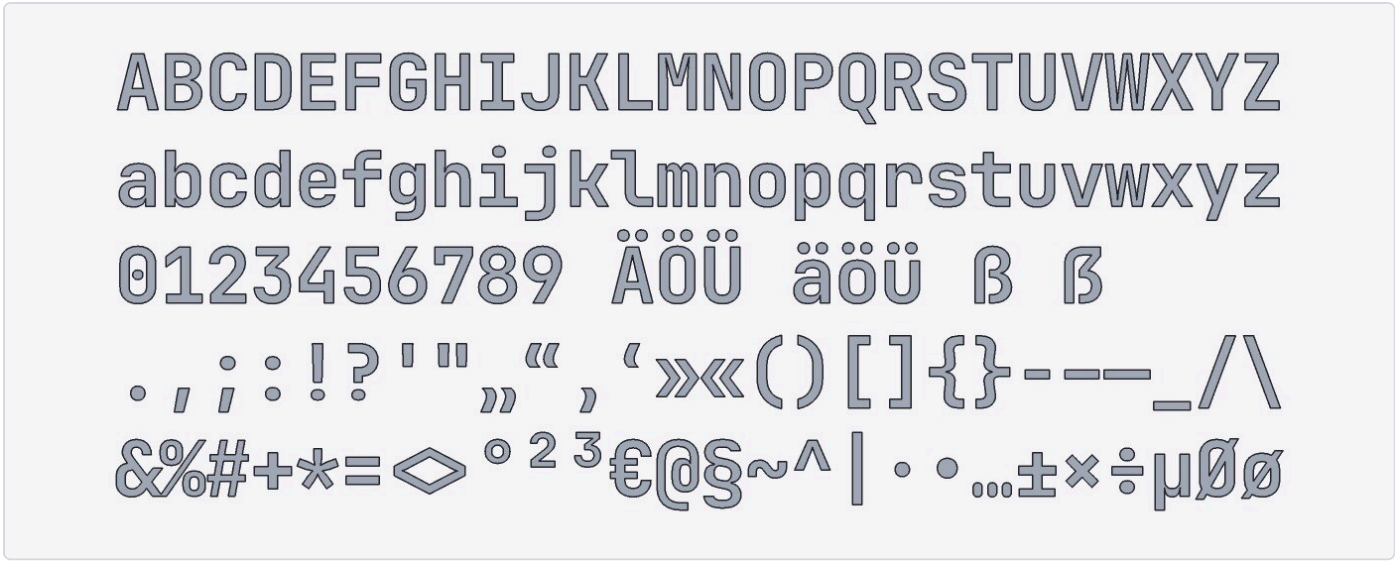
Mitgeliefert in `crates/cad-font/fonts/` (SIL OFL 1.1, Lizenztexte und Quellen mit Prüfsummen in `fonts/README.md`), eingebettet mit `include_bytes!`: `AllertaStencil-Regular.ttf` 20 KiB, `Inter-Bold.ttf` 411 KiB, `Inter-Regular.ttf` 402 KiB, `JetBrainsMono-Bold.ttf` 271 KiB, `JetBrainsMono-Regular.ttf` 267 KiB. Auswahl per Familie und Stil wie OpenSCAD (`"Inter:style=Bold"`), eigene Schriften per Pfad (`font = "datei.ttf"`) bzw. Upload in der UI (T03).

Alphabet-Tafeln

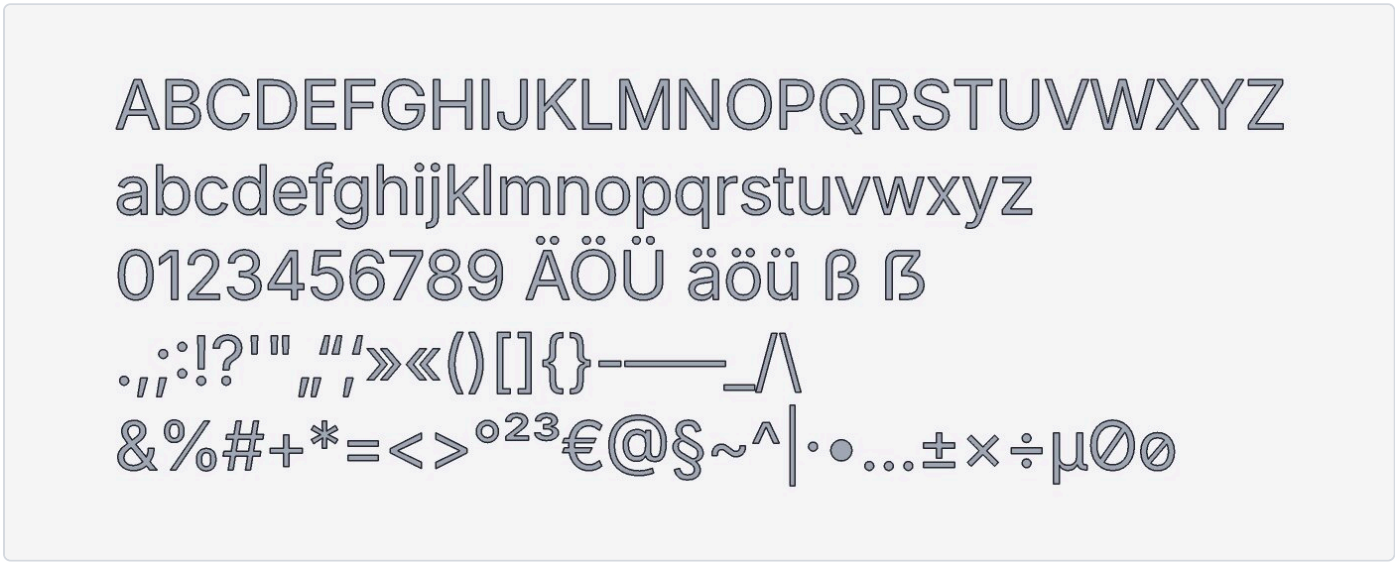
Je Schrift `text(…, size = 8)`, 0,6 mm extrudiert, von oben (Generator `tools/report/report16-data.sh`). Zeile 3–5: Ziffern, **Umlaute und Satzzeichen** des Pflicht-Zeichensatzes.



JetBrains Mono (Standard) – JetBrains Mono 2.304, Regular



JetBrains Mono:style=Bold – JetBrains Mono 2.304, Bold



Inter – Inter 4.1, Regular



Inter:style=Bold – Inter 4.1, Bold



Allerta Stencil – Allerta Stencil (Google Fonts), Regular

Pflicht-Zeichensatz

`cad_font::REQUIRED_CHARS` (Test `required_chars_in_every_font`: jede mitgelieferte Schrift hat für jedes Zeichen eine echte Glyphe, kein `.notdef`): A–Z a–z 0–9, Ä Ö Ü ä ö ü ß ß, Leerzeichen, `. , ; : ! ? ' " „ “ , ‘ » « ( ) [ ] { } - – — _ / \ & % # + * = < > ° ² ³ € @ § ~ ^ | · • …, ± × ÷ µ Ø ø`. Gilt auch für die künftige Schablonenschrift (Plan 17.4).

Schrift	fehlende Zeichen	Folge
JetBrains Mono Regular/Bold	keine	–
Inter Regular/Bold	keine	–
Allerta Stencil	`ß „ , { } _ # ° ² ³ § ~ ^ · • … ± µ Ø ø`	aus JetBrains Mono Regular, Warnung (siehe Tafel: diese Zeichen sind nicht schabloniert)

```
cases/fallback.scad:1:24: Warnung (Auswertung): Zeichen „°±“ fehlen in Allerta Stencil; ersetzt aus JetBrains Mono Regular
cases/fallback.scad: 0 Fehler, 1 Warnungen
```

4 Text gegen OpenSCAD 2021.01

Gleiche Schriftdateien (OpenSCAD über eine fontconfig-Datei, die nur `crates/cad-font/fonts` kennt), `text(…)` als 2D exportiert (Polygone) gegen `cad measure --bounds --volume` einer 1-mm-Extrusion (`tools/report/report16-openscad.py`). Hülle in x (oben) und y (unten), mm.

Fall	OpenSCAD Hülle	cad Hülle	max. Abw.	OpenSCAD mm²	cad mm²	Fläche
Inter 'H'	1,221 … 9,101 0,000 … 10,105	1,221 … 9,101 0,000 … 10,105	0,0000	32,082	32,082	0,002 %
Inter:style=Bold 'H'	0,915 … 9,460 0,000 … 10,105	0,916 … 9,460 0,000 … 10,105	0,0000	49,421	49,422	0,001 %
JetBrains Mono 'H'	1,292 … 7,041 0,000 … 10,138	1,292 … 7,042 0,000 … 10,139	0,0007	29,044	29,049	0,015 %
JetBrains Mono:style=Bold 'Hamburg'	1,069 … 56,091 -2,500 … 10,138	1,069 … 56,092 -2,500 … 10,139	0,0007	235,350	235,737	0,164 %
Allerta Stencil 'H'	1,275 … 8,883 0,000 … 9,996	1,275 … 8,884 0,000 … 9,996	0,0006	37,000	37,005	0,014 %

Inter 'AV'	0,353 … 17,661 0,000 … 10,105	0,353 … 17,661 0,000 … 10,105	0,0001	54,797	54,799	0,004 %
Inter 'O'	0,827 … 9,793 -0,136 … 10,240	0,827 … 9,793 -0,136 … 10,240	0,0001	33,163	33,257	0,283 %
Inter 'Hello World'	1,221 … 73,349 -0,163 … 10,105	1,221 … 73,349 -0,163 … 10,105	0,0000	229,351	229,494	0,062 %
Inter 'Größe'	0,827 … 38,910 -0,163 … 10,430	0,827 … 38,910 -0,163 … 10,430	0,0000	130,494	130,710	0,166 %
Allerta Stencil 'ZUTRITT'	1,058 … 56,991 -0,068 … 9,996	1,058 … 56,991 -0,068 … 9,996	0,0006	185,418	185,490	0,039 %
JetBrains Mono '→'	0,986 … 15,429 0,486 … 8,680	0,985 … 15,430 0,486 … 8,681	0,0006	26,073	26,076	0,009 %
Inter 'fi'	0,136 … 7,563 0,000 … 10,579	0,136 … 7,563 0,000 … 10,579	0,0001	29,216	29,234	0,062 %
Inter 'H', halign = "center", valign = "center"	-3,819 … 4,061 -4,934 … 5,170	-3,819 … 4,061 -4,934 … 5,171	0,0004	32,082	32,082	0,002 %
Inter 'Hg', valign = "bottom"	1,221 … 17,526 -0,066 … 13,036	1,221 … 17,526 -0,070 … 13,032	0,0040	66,071	66,052	0,029 %
Inter 'Hg', valign = "top"	1,221 … 17,526 -12,866 … 0,236	1,221 … 17,526 -12,865 … 0,237	0,0009	66,071	66,052	0,029 %
Inter 'HH', spacing = 2	1,221 … 29,261 0,000 … 10,105	1,221 … 29,261 0,000 … 10,105	0,0000	64,164	64,165	0,002 %
Inter 'HH', direction = "ttb"	-3,819 … 4,061 -29,551 … -3,035	-3,819 … 4,061 -29,551 … -3,035	0,0001	64,164	64,165	0,002 %
Inter 'H', halign = "right"	-8,859 … -0,979 0,000 … 10,105	-8,859 … -0,979 0,000 … 10,105	0,0000	32,082	32,082	0,001 %
Inter 'Text', size = 25	1,662 … 68,083 -0,407 … 25,262	1,662 … 68,083 -0,407 … 25,262	0,0001	503,680	504,016	0,067 %

Die Fläche von OpenSCAD ist kleiner, weil es die Kurven als Polygone abtastet (Sehnen innerhalb der Kurve); die Hülle stimmt bis auf ≤ 0,001 mm (OpenSCAD schreibt 6 Stellen; seine Polygonecken liegen auf der Kurve). `valign` misst OpenSCAD an gerasterten Glyphenmetriken (FreeType), daher dort bis 0,004 mm. „Hello World“, „AV“ (Kerning), „fi“ und „→“ (Ligaturen in JetBrains Mono) zeigen, dass Shaping und Laufweite gleich sind.

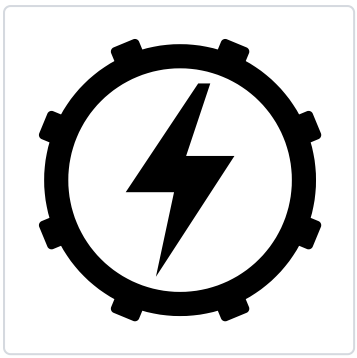
5 SVG-Import

5.1 Beispiele: Quelle und Ergebnis

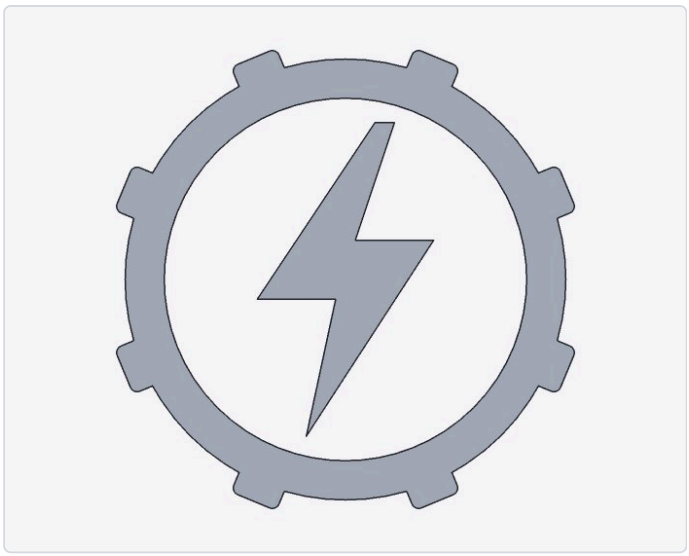
Links die SVG-Datei, wie der Browser sie zeichnet (Quelle in `examples/svg/`, eigene Zeichnungen), rechts `linear_extrude(2) import("….svg")` von oben.

logo.svg

Ring aus Halbkreisbögen mit Loch (evenodd), acht gedrehte Rechtecke mit Rundung, Polygon



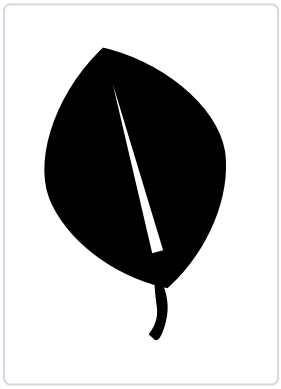
Quelle (Browser)



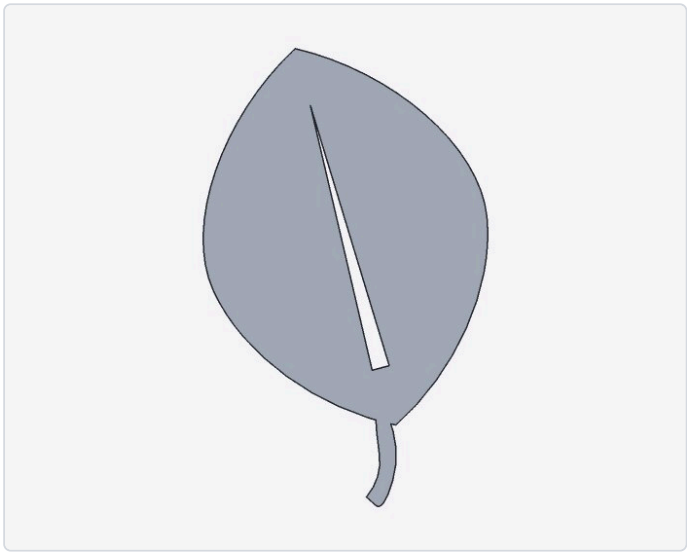
import, extrudiert, von oben

blatt.svg

kubische Bézier (C, S), Rippe als Gegenlauf (nonzero), Stiel aus Q/T, gedrehte Gruppe



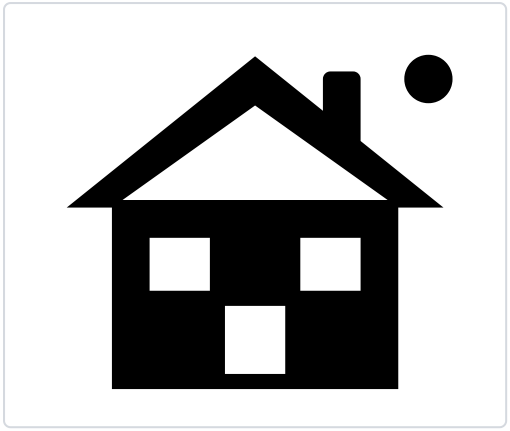
Quelle (Browser)



import, extrudiert, von oben

haus.svg

relative Befehle (h, v, m), Fenster und Tür als Löcher (evenodd), Rechteck mit rx, skalierte Gruppe mit Kreis



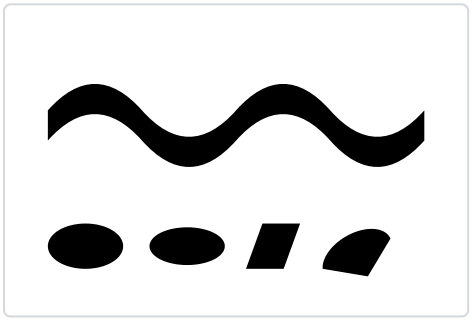
Quelle (Browser)



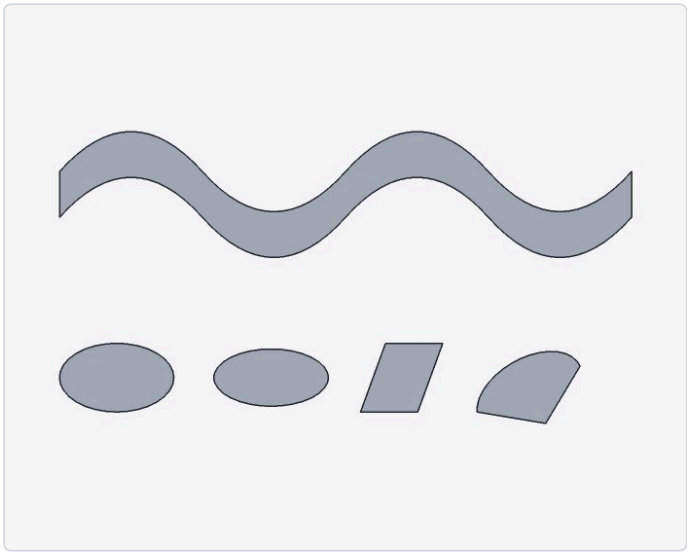
import, extrudiert, von oben

welle.svg

Q/T-Welle, Ellipse, Kreis mit scale(2 1) (Ellipsenbögen), skewX, gedrehter Ellipsenbogen (A)



Quelle (Browser)



import, extrudiert, von oben

5.2 Unterstützt und nicht unterstützt

Element / Eigenschaft	Umsetzung
`path`	M L H V C S Q T A Z, absolut und relativ, implizite Wiederholung, kompakte Zahlen (`10-10.5.5`) und Bogen-Flags (`a10 10 0 0110 10`); Fehler: der Teil davor bleibt, Warnung
Bögen `A`, `circle`, `ellipse`, `rect rx/ry`	exakte Kreisbögen (unter Ähnlichkeiten), sonst exakte Ellipsenbögen (rationale B-Spline); Radien zu klein → hochskaliert wie SVG
`polygon` · `polyline`, `line`, offene Pfade	Fläche · nur Pfad (Warnung „offener Pfad wird nicht gefüllt“); nur Pfade → offene Skizze (als `sweep`-Pfad)
`g`, `a`, verschachteltes `svg`, `transform`	matrix, translate, scale, rotate (auch um Punkt), skewX, skewY; Listen von rechts nach links
`viewBox`, `width/height`, `preserveAspectRatio`	px, mm, cm, in, pt, pc, ohne Einheit (`dpi`); Ursprung unten links
`fill`, `fill-rule`, `display`, `visibility`	als Attribut und in `style`, vererbt; `fill="none"`: nicht übernommen (Warnung)

Text, Bilder, `use`, CSS-Klassen/`<style>`, Masken, Clip-Pfade, Filter, Verläufe/Muster als Füllung, Strichstärke	Warnung mit SVG-Zeile (je Meldung einmal); Verläufe werden als einfache Fläche übernommen
Metadaten (Inkscape, Sodipodi, `defs`, `title`)	stillschweigend übergangen
`id = "…"`	nur dieses Element (mit Inhalt); unbekannt: Fehler

```
<svg xmlns="http://www.w3.org/2000/svg" width="40mm" height="20mm" viewBox="0 0 40 20">
  <style>.a { fill: red }</style>
  <rect class="a" width="20" height="20"/>
  <text x="22" y="12">Hallo</text>
  <image href="foto.png" width="10" height="10"/>
  <path d="M 22 2 L 38 2 L 38 8" stroke="black" fill="none"/>
  <use href="#r"/>
</svg>
```

```
cases/warn.scad:1:26: Warnung (Auswertung): warn.svg:2: CSS-Stylesheet (`<style>`) nicht unterstützt
cases/warn.scad:1:26: Warnung (Auswertung): warn.svg:3: `rect`: CSS-Klassen (`class`) nicht unterstützt; Stil
nur aus Attributen und `style`
cases/warn.scad:1:26: Warnung (Auswertung): warn.svg:4: `text`: Text nicht unterstützt (in Pfade umwandeln)
cases/warn.scad:1:26: Warnung (Auswertung): warn.svg:5: `image`: Bilder nicht unterstützt
cases/warn.scad:1:26: Warnung (Auswertung): warn.svg:6: `path`: offener Pfad wird nicht gefüllt (nur als Pfad
übernommen; Z schließt ihn)
cases/warn.scad:1:26: Warnung (Auswertung): warn.svg:6: Strichstärke (`stroke`) wird nicht als Fläche
übernommen
cases/warn.scad:1:26: Warnung (Auswertung): warn.svg:7: `use`: Verweise nicht unterstützt (Klone in Pfade
umwandeln)
cases/warn.scad:1:26: Warnung (Auswertung): warn.svg: 1 offene Pfade neben Flächen weggelassen
cases/warn.scad: 0 Fehler, 8 Warnungen
```

5.3 Einheiten und Vergleich mit OpenSCAD

OpenSCAD 2021.01 (`import(…, dpi = 96)`, 2D exportiert) gegen cad. Mit `viewBox` stimmen beide überein. Abweichungen mit Absicht (SVG-Regel statt OpenSCAD-Eigenheit): **ohne `viewBox`** nimmt OpenSCAD Nutzereinheiten als mm, spiegelt aber um die Höhe in Pixeln (Zeile „ohne-viewbox“); ein y-Versatz der `viewBox` wird bei OpenSCAD addiert statt abgezogen; OpenSCAD füllt immer nach evenodd und macht aus Strichen Flächen.

Datei	OpenSCAD Hülle	cad Hülle	max. Abw.	OpenSCAD mm²	cad mm²	Fläche
logo.svg	1,311 … 38,689 1,311 … 38,689	1,311 … 38,689 1,311 … 38,689	0,0006	492,531	492,893	0,073 %
blatt.svg	2,039 … 26,136 2,551 … 41,319	2,033 … 26,141 2,549 … 41,319	0,0055	536,447	536,858	0,077 %
haus.svg	5,000 … 56,200 2,000 … 46,200	5,000 … 56,200 2,000 … 46,200	0,0000	1.075,692	1.075,832	0,013 %
welle.svg	2,500 … 52,500 2,000 … 27,500	2,500 … 52,500 2,000 … 27,500	0,0000	352,785	353,326	0,153 %
mm-viewbox	5,000 … 29,000 36,000 … 47,500	5,000 … 29,000 36,000 … 47,500	0,0000	100,059	100,265	0,207 %
px-viewbox	2,646 … 15,346 5,821 … 11,906	2,646 … 15,346 5,821 … 11,906	0,0000	28,018	28,076	0,207 %
in-viewbox	2,540 … 14,732 5,588 … 11,430	2,540 … 14,732 5,588 … 11,430	0,0000	25,822	25,875	0,206 %
ohne-viewbox	10,000 … 58,000 -14,771 … 8,229	2,646 … 15,346 5,821 … 11,906	42,6542	400,236	28,076	92,985 %

6 Beispiele

examples/spec/text_schild.scad

```
// Zweck: Schild mit Relief-Text (Plan 17.4, Bericht
16) – Grundplatte mit Rand, erhabener Text in
// zwei Schriften – mehrfarbig (3MF mit einem
Filament je Farbe).
// Schriften: mitgeliefert sind JetBrains Mono
(Standard), Inter und Allerta Stencil, je mit
// Stil (`"Inter:style=Bold"`); eigene Schriften mit
`font = "datei.ttf"` (Datei im Projekt).
// `size` wie in OpenSCAD: Versalhöhe etwa `size`.

breite = 100;
hoehe = 44;
dicke = 3;
r = 5;          // Eckenradius
rand = 2;       // Breite des Rands
relief = 1.2;   // Höhe von Text und Rand

module platte(b, h, r) sketch {
    start([r, 0]);
    line(to = [b - r, 0]);
    arc(center = [b - r, r], angle = 90);
    line(to = [b, h - r]);
    arc(center = [b - r, h - r], angle = 90);
    line(to = [r, h]);
    arc(center = [r, h - r], angle = 90);
    line(to = [0, r]);
    arc(center = [r, r], angle = 90);
}

color("white") linear_extrude(dicke) platte(breite,
hoehe, r);

// Rand
color("blue") translate([0, 0, dicke])
linear_extrude(relief) difference() {
    platte(breite, hoehe, r);
    translate([rand, rand]) platte(breite - 2 * rand,
hoehe - 2 * rand, r - rand);
}

// Titel und Untertitel erhaben
color("red") translate([breite / 2, 26, dicke])
    linear_extrude(relief) text("Werkstatt", size =
11, font = "Inter:style=Bold", halign = "center",
valign = "center");
color("black") translate([breite / 2, 9, dicke])
    linear_extrude(relief) text("ZUTRITT NUR MIT
BRILLE", size = 4.2, font = "Allerta Stencil", halign
= "center");
```



Schild: Platte aus einer Skizze, Rand, erhabener Text in Inter Bold und Allerta Stencil, vier Farben



von oben

```
Farben (Filament, Farbe, Volumen, Masse PLA 1,24
g/cm³):
  F1  white          #f4f3f0  13135.6195 mm³  16.29 g
Z.26
  F2  blue           #2e65b9  655.5186 mm³  0.81 g
Z.29
  F3  red            #c74240  445.7077 mm³  0.55 g
Z.35
  F4  black          #202225  117.2133 mm³  0.15 g
Z.37
```

examples/spec/svg_logo.scad

```
// Zweck: SVG-Import (Plan 17.5, Bericht 16) – Logos
und Icons aus examples/svg als exakte 2D-Profile.
// Kreise und Bögen bleiben Kreisbögen, Bézierkurven
bleiben Kurven, Löcher nach fill-rule.
// Einheiten: viewBox + width/height in mm; ohne
viewBox Pixel mit dpi (Standard 96).

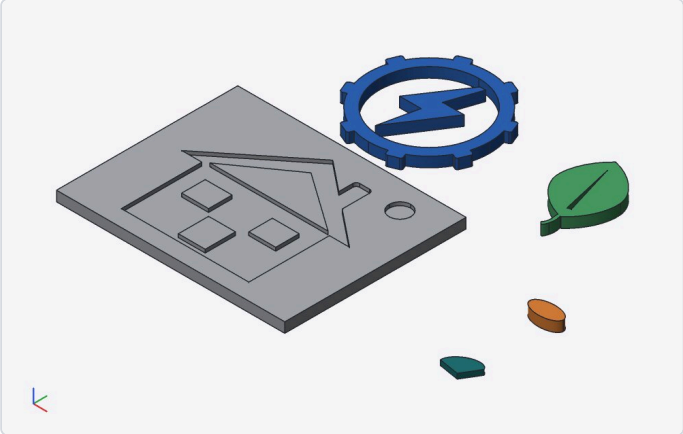
dicke = 3;

// Logo, mittig um den Ursprung (center = true)
color("blue") linear_extrude(dicke)
import("../svg/logo.svg", center = true);

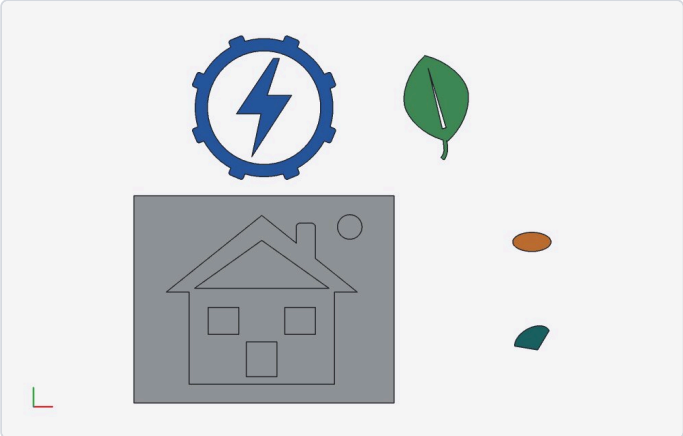
// Blatt, auf 70 % verkleinert
translate([45, 0, 0])
    color("green") linear_extrude(dicke)
import("../svg/blatt.svg", center = true, scale =
0.7);

// Haus als Prägung in einer Platte
translate([0, -50, 0]) difference() {
    color("silver") translate([-34, -27, 0])
cube([68, 54, dicke]);
    color("red") translate([0, 0, dicke - 1])
linear_extrude(2) import("../svg/haus.svg", center =
true);
}

// Nur einzelne Elemente des SVG (id): ungleichmäßig
skalierter Kreis (exakte Ellipsenbögen),
// gedrehter Ellipsenbogen
translate([70, -35, 0]) color("orange")
linear_extrude(dicke) import("../svg/welle.svg", id =
"oval", center = true);
translate([70, -60, 0]) color("teal")
linear_extrude(dicke / 2) import("../svg/welle.svg",
id = "bogen", center = true);
```



Logo, Blatt (skaliert), Haus als Prägung, aus welle.svg nur id = "oval" und id = "bogen"



von oben

STEP von svg_logo: {"CIRCLE": 78, "LINE": 253, "ELLIPSE": 4, "B_SPLINE_CURVE_WITH_KNOTS": 27, "PLANE": 85, "CYLINDRICAL_SURFACE": 39, "SURFACE_OF_LINEAR_EXTRUSION": 13, "B_SPLINE_SURFACE_WITH_KNOTS": 0} – Kreise bleiben `CIRCLE`, extrudierte Bögen `CYLINDRICAL_SURFACE`.

7 Messwerte

Kernel-Zeit

Release-Build, `cad eval --json` ohne Daemon, Median aus 3 Läufen (Sprache = Auswertung inkl. Shaping und SVG; Kernel = alle Kernel-Aufrufe). Browser: dieselben Modelle in Chromium (SMLib als Wasm, T05).

Modell	Knoten	Kernel-Aufrufe	Sprache ms	Kernel ms nativ	Kernel ms Browser
„Hello World“ (Inter), extrudiert	13	20	0,7	505	918
40 Zeichen (Inter), extrudiert	36	61	1,1	2.249	2.904
40 Zeichen (JetBrains Mono)	36	61	1,2	1.917	–
„Hello World“ auf einer Platte (Vereinigung)	15	24	0,7	1.449	–
examples/spec/svg_logo.scad	41	45	0,8	1.429	–
examples/spec/text_schild.scad	50	66	1,3	10.550	–

Je Glyphe: Extrusion je Kontur (Außen, Loch), Differenz für Löcher, eine Vereinigung aller Glyphen (sie berühren sich nicht). Teuer ist Text **auf** einer Fläche: SMLib prägt jede Glyphe in die Deckfläche (Schild: eine Vereinigung

von ≈ 9 s nativ). Die neue Farbregion-Regel (Plan 17.10) vermeidet dabei die Booleans mit dem Ganzen; vorher brauchte der Kontaktbogen von T01 nativ 21 s und scheiterte („Schnittmenge gescheitert“), jetzt 4 s.

Wasm-Modul

	roh	gzip -9
`cad_wasm_bg.wasm` vorher (Commit cafe7a4)	1.799.439 B	702.133 B
mit Text/SVG (vor dem Merge von Schritt 2)	3.970.940 B	1.696.909 B
nach dem Merge mit Skizzen Schritt 2 (Stand dieses Berichts)	4.120.239 B	1.757.298 B
Zuwachs durch Text/SVG	+2.171.501 B	+994.776 B
davon Schriftdateien	1.404.056 B	–

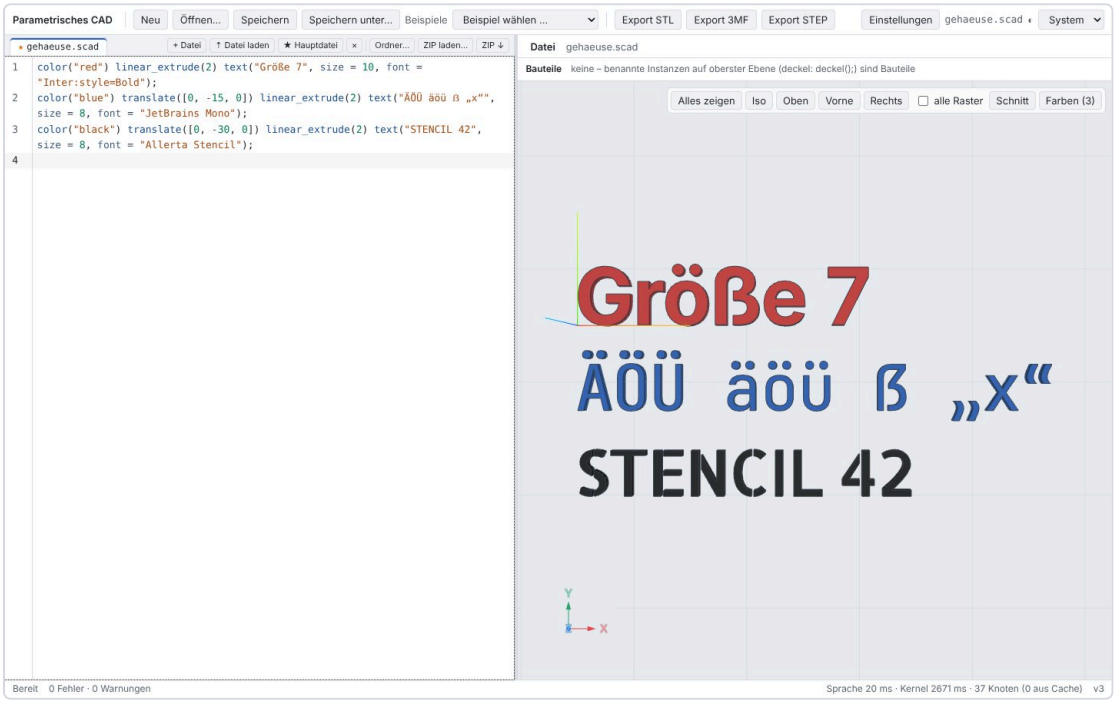
Der Rest des Zuwachses ist Code (rustybuzz/ttf-parser, roxmltree, SVG- und Textlogik). `smlib.wasm`: 5.285.361 B (+726 B durch `disjoint_union` in der Fassade).

8 UI

Prüfungen über den Zustand der UI (`window.__cad`), Szenen und Hüllen; Screenshots aus `web/e2e/text-svg.spec.ts`.

T01 Text in drei Schriften

Umlaute, ß und deutsche Anführungszeichen ohne Warnung; Farben wie sonst.



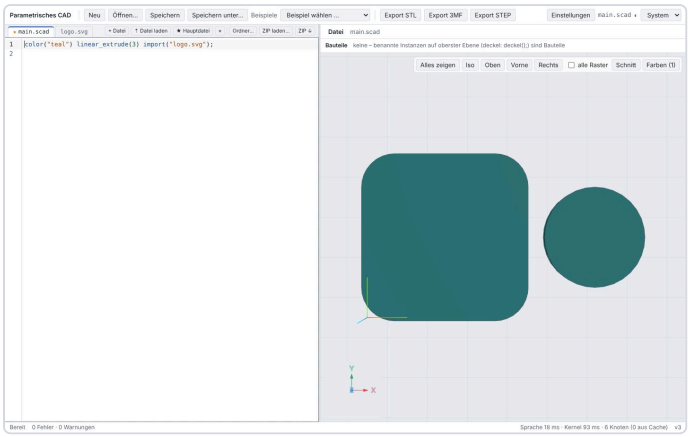
Inter Bold, JetBrains Mono, Allerta Stencil (Draufsicht)

```
color("red") linear_extrude(2) text("Größe 7", size = 10, font = "Inter:style=Bold");
color("blue") translate([0, -15, 0]) linear_extrude(2) text("ÄÖÜ äöü ß „x“", size = 8, font = "JetBrains
Mono");
color("black") translate([0, -30, 0]) linear_extrude(2) text("STENCIL 42", size = 8, font = "Allerta Stencil");
```

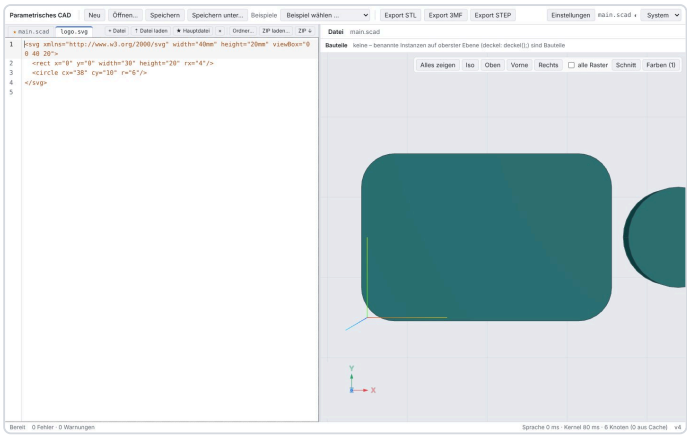
- keine Fehler/Warnungen
- Breite > 50 mm, Höhe 2 mm
- „20°C“ in Allerta Stencil: Warnung, ° aus JetBrains Mono

T02 SVG als Projektdatei, im Editor geändert

Das SVG ist eine Projektdatei (Tab), wird im Editor geändert; die Szene rechnet neu (Cache-Schlüssel aus dem Inhalt).



Rechteck 20 mm + Kreis



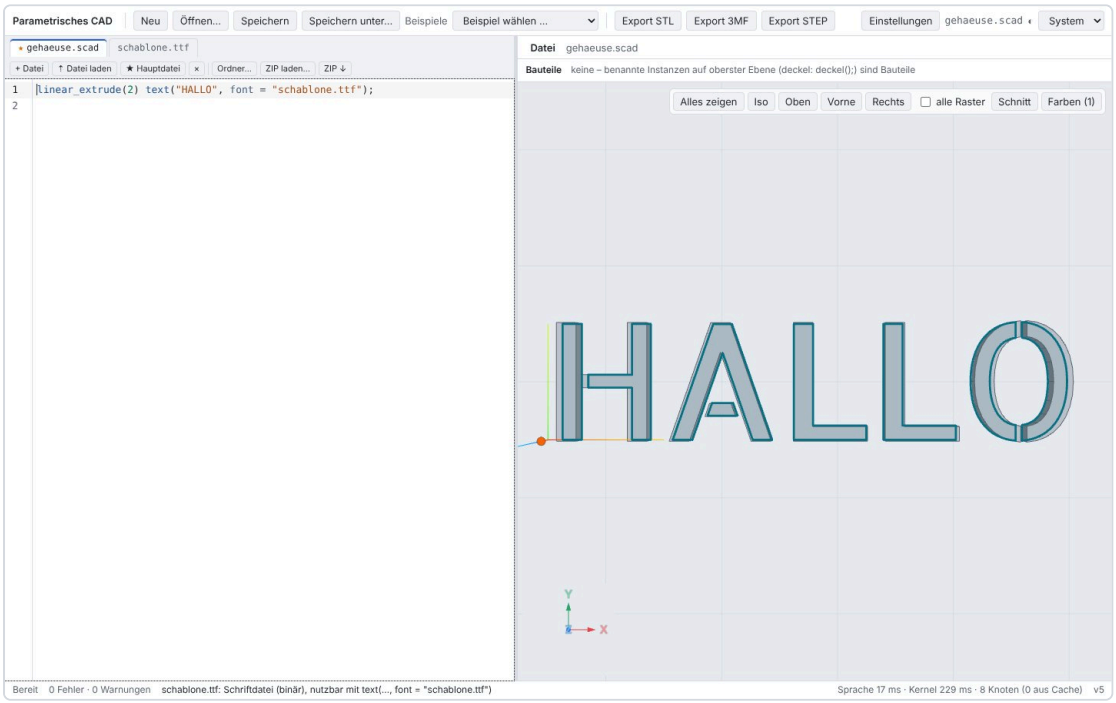
logo.svg im Editor: Rechteck 30 mm

```
color("teal") linear_extrude(3) import("logo.svg");
```

- Breite 34 mm
- nach der Änderung 44 mm

T03 Eigene Schrift (Upload)

Binärdateien liegen im Projekt als data-URL; die Engine dekodiert sie. Der Tab öffnet keinen Editor.

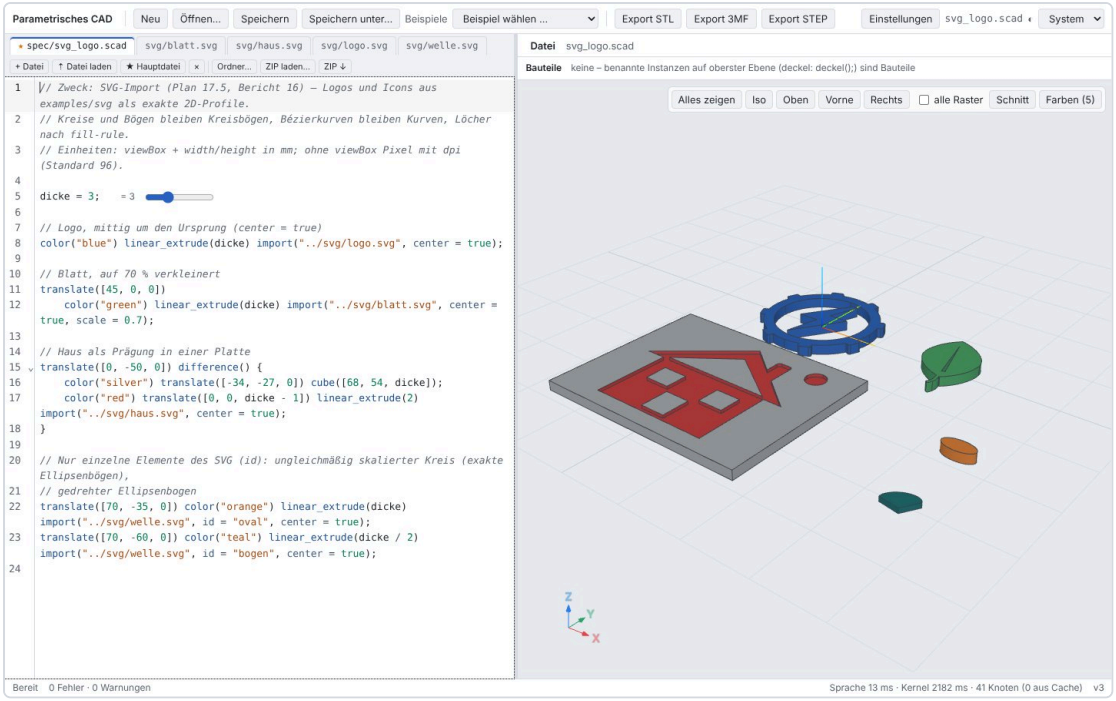


„↑ Datei laden“: schablone.ttf, text(..., font = "schablone.ttf")

- Datei im Projekt
- keine Warnung
- Tab: Hinweis statt Editor

T04 Beispiel mit SVG-Dateien

Einzelbeispiele, die SVGs aus examples/svg importieren, öffnen als Projekt mit diesen Dateien.



examples/spec/svg_logo.scad als Projekt mit svg/*.svg

- 4 SVG-Dateien im Projekt
- keine Fehler
- <text> im SVG: Warnung „w.svg:3: …Text…“

9 Testläufe

Lauf	Ergebnis
`cargo test --workspace --no-fail-fast`	520 bestanden, 0 fehlgeschlagen
`cargo clippy --workspace --all-targets -- -D warnings` · `cargo fmt --check`	sauber · sauber
`pnpm typecheck` · `pnpm test` (Vitest)	sauber · 220 bestanden
`pnpm e2e` (Playwright, Chromium)	102 / 103 – fehlgeschlagen: C16 Punktgriff achsgebunden ziehen (Umschalttaste, X/Y/Z) · C16 (Punktgriff achsgebunden ziehen, Bericht 3) ist zeitabhängig (so schon in Bericht 15): im Gesamtlauf und einzeln mit -g C16 Test-Timeout, die ganze Datei ui-c.spec.ts danach 20/20 bestanden – auch mit der Fassade vor diesem Schritt einzeln gescheitert, also nicht von Text/SVG berührt.
`tests/openscad-compat/compare.py --eval`	36/36
`tests/step/verify.py` (OpenCASCADE)	60/60 passed
`tests/alltag/check.sh`	19 Modelle, 0 mit Problemen, 56 s, Exporte in /tmp/alltag-2sczuntc (gelöscht)
`node crates/kernel-smlib/js/test-node.mjs`	bestanden

Neue bzw. geänderte Rust-Tests (42):

`import_volume_warning_and_step` **ok** · `watch_follows_a_changed_svg` **ok** · `text_volume_is_area_times_height` **ok** · `examples_check_clean` **ok** · `data_urls_roundtrip` **ok** · `import_open_paths_are_a_path` **ok** · `import_svg_gives_sketch_profiles` **ok** · `changed_svg_changes_the_tree` **ok** · `import_relative_to_the_file_and_errors` **ok** · `text_with_a_font_file` **ok** · `text_profiles_and_fonts` **ok** · `unsupported_operations_keep_children` **ok** · `tests::fallback_to_jetbrains_mono` **ok** · `tests::font_names` **ok** · `tests::glyph_o_has_a_hole` **ok** · `tests::directions` **ok** · `tests::align_and_spacing` **ok** ·

`tests::kerning_pair_av` **ok** · `tests::cff_outlines_are_cubic` **ok** · `tests::user_font_data` **ok** · `tests::hull_and_area_like_openscad` **ok** · `tests::required_chars_in_every_font` **ok** · `tests::normalize_rules` **ok** · `tests::circle_area_and_mirror` **ok** · `path::tests::arcs_exact` **ok** · `path::tests::errors` **ok** · `path::tests::commands_absolute_relative` **ok** · `path::tests::implicit_lineto_and_compact_numbers` **ok** · `path::tests::smooth_curves_reflect` **ok** · `tests::nested_svg_viewport` **ok** · `tests::hidden_and_unfilled` **ok** · `tests::errors` **ok** · `tests::open_paths_are_paths` **ok** · `tests::fill_rules` **ok** · `tests::select_by_id` **ok** · `tests::units_viewbox_like_openscad` **ok** · `tests::transforms` **ok** · `tests::shapes_exact` **ok** · `tests::units_without_viewbox_are_pixels` **ok** · `tests::unsupported_warns_with_line` **ok** · `tests::transforms_before_grid` **ok** · `spec_models_stage1` **ok**

SVG (`cad-svg`): Pfadbefehle, Bögen und Flags, Transformationen, Einheiten mit OpenSCAD-Referenzwerten, Füllregeln, verborgene/ungefüllte Elemente, Warnungen mit Zeile, `id`, Fehlerfälle. Text (`cad-font`): Hülle und Fläche gegen OpenSCAD (20 Fälle), Glyphe „O“ mit Loch, Kerning „AV“, Ausrichtung, Richtungen, Schriftnamen, eigene Schrift, CFF-Umrisse, Pflicht-Zeichensatz, Rückfall. `cad-eval`: Profile, Projektdateien, Fehler, data-URLs. CLI: Beispiele, Volumen, STEP-Kreis, `watch` folgt dem SVG. Vitest: Datendateien (`web/src/logic/files.test.ts`).

10 Grenzen

- Eigene Schablonenschrift fehlt (Schritt 3, nach `stroke`); Allerta Stencil hat 20 Zeichen des Pflicht-Zeichensatzes nicht (aus JetBrains Mono, nicht schabloniert).
- Kernel-Zeit: viele Glyphen auf einer gemeinsamen Fläche sind teuer (Vereinigung in SMLib wächst überlinear: 9 Glyphen ≈ 1,5 s, 28 ≈ 9 s nativ); im Browser dauert es etwa 1,3- bis 1,8-mal so lange, ein einzelner Kernel-Aufruf über 15 s löst den Watchdog aus.
- Füllregeln setzen sich nicht schneidende Konturen innerhalb eines Elements voraus (Schriften mit überlappenden Konturen, z. B. manche Variable Fonts, und sich selbst schneidende SVG-Pfade werden nicht aufgelöst). Variable Fonts nur in der Standardinstanz.
- SVG: kein Text, keine `use`-Klone, keine CSS-Klassen, keine Striche als Fläche, keine Masken/Clip-Pfade; Prozent- und em-Längen werden wie Zahlen behandelt.
- Dateinamen müssen als Text im Projekt stehen (ein zur Laufzeit zusammengesetzter Name wird nicht geladen).
- Gefunden beim Logo: Vereinigungen extrudierter Skizzenbögen über 90° rechnete die Fassade falsch (Skizzen Schritt 1); behoben in Schritt 2 (`profile::split_arcs`), die zwischenzeitliche Umgehung im Import entfiel.
- STEP: Extrusionen von Ellipsenbögen schreibt der Writer jetzt als Extrusion einer `ELLIPSE` (OpenCASCADE hatte die rationale B-Spline um $3{,}6 \cdot 10^{-3}$ falsch integriert). Offen: die Extrusion der quadratischen Q/T-Welle aus `welle.svg` misst OpenCASCADE um 3 % zu klein (Kernel 652,25 mm², analytisch 652,19 mm²) – `svg_logo.scad` nimmt aus `welle.svg` daher nur Ellipse und Bogen; die Welle selbst steht in 5.1.
- Standardausrichtung senkrechten Texts wie OpenSCAD (Versatz des Shapers); `valign` nach der echten Glyphenhülle (OpenSCAD: gerasterte FreeType-Metriken, ≤ 0,005 mm).